OOP Scenario-Based Assignment

Online Banking Account Management System

Business Scenario

You are part of a development team building a **core banking module** for an online banking system.

The bank wants a **secure and extensible design** to manage different types of bank accounts and transactions.

The system must follow **strict Object-Oriented principles**:

- Encapsulation
- Responsibility segregation
- Reusability
- Clean design

PART 1: Account Design

1 Create a class `BankAccount`

Data Members (MUST be `private`)

Field	Type	Description
accountNumber	long	Unique account number
accountHolderName	String	Customer name
balance	double	Current balance
isActive	boolean	Account status

---

2 Constructor Rules

- Create a **parameterized constructor** to initialize:
  - `accountNumber`
  - `accountHolderName`
  - `balance`
- New account is **active by default**

---

3 Encapsulation Rules (STRICT)

- `balance` **must NOT be modified directly**
- Only **business methods** can change balance
- Getter allowed for balance
- Setter for balance is ✕NOT allowed

PART 2: Business Operations

4 Implement the following methods in `BankAccount`

```
public void deposit(double amount)
public boolean withdraw(double amount)
public void deactivateAccount()
public boolean isAccountActive()
```

Business Rules:

- Deposit amount must be > 0
- Withdrawal allowed only if:
  - Account is active
  - Sufficient balance exists
- Withdrawal returns `true` if successful, else `false`
- No transaction allowed if account is inactive

PART 3: Transaction Responsibility

5 Create a separate class `TransactionService`

IMPORTANT DESIGN RULE
`TransactionService` must NOT access `balance` directly
It must use **public methods of `BankAccount`**

Responsibilities:

- Perform deposit
- Perform withdrawal
- Display transaction result messages

□ PART 4: Constructor + Setter Logic (Realistic)

6□ Account Update Requirement

- Allow customer to:
  - Change account holder name
- Use **setter method**
- Account number must NEVER change

□ PART 5: Application Layer

7□ Create a class `BankApp`

In `main()`:

1. Create **multiple BankAccount objects**
2. Perform:
   - Deposit
   - Withdrawal
   - Invalid withdrawal
   - Deactivate account
   - Attempt transaction on inactive account
3. Display final account details using **getters only**

□ Sample Output (Indicative)

```
Account Created Successfully
Deposited: 5000
Withdrawal Successful: 2000
Insufficient Balance
Account Deactivated
Transaction Failed: Account is inactive

Final Balance: 3000
```